

AWS Module 2 :

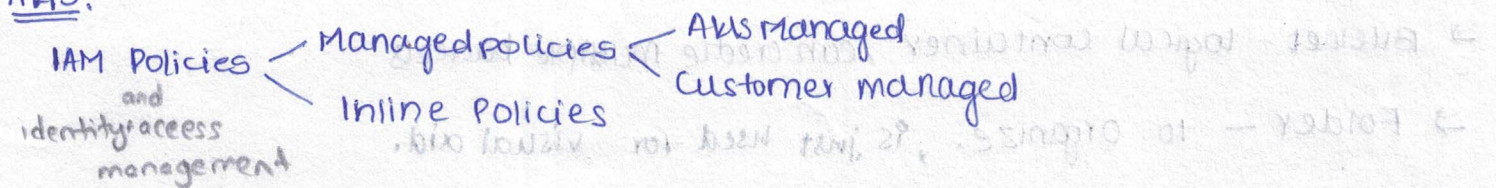
- save money — pay for what you use
- get volume discount
- custom pricing for large volumes
- Big companies

→ For new user — gives a free tier

Total cost of ownership : on premise Vs Cloud

CapEx - yes	only OpEx ⇒ depends on service use	→ price Calculator → estimate monthly costs etc...
----------------	---------------------------------------	---

AWS:



Best practices

- * shouldn't share login credentials → create user and provide their own account
- * create groups — Assign permission rather than individuals
- * Permissions should be considered — give the least privileges
- * Auditing — cloud trail
- * good password policy, change frequently password (rotate passwords)
- * MFA for every user.
Multi Factor authentication
- * Root user shouldn't be frequently used. — create a self IAM user.

AWS core services :

- storage — S3, EBS, etc...
- Compute — EC2, Lambda, lightsail etc...
- Database — RDS, DynamoDB, Redshift
- Network — VPN, Direct Connect etc...

scenario: Architecture of an APP using AWS:

The final project

Storage Services

S3 : — present outside VPC, but accessed by VPC (Setup required)

→ simple storage service

→ store, retrieve, access & backup any amt of data, from any where

→ accessed from AWS management console, CLI, SDKs

⇒ Bucket — logical container, can create multiple buckets

⇒ Folder — to organize, is just used for visual aid.

⇒ S3 doesn't use hierarchical system.

⇒ S3 Object — any file in S3, max size of object — 5TB, if exceeded, object broken up into chunks & should be uploaded.

↳ Object versioning — retaining of versions

→ storage class — classes according to frequency of access

⇒ A Resource based policy.

x) Bucket name should be unique

⇒ Important concepts of Storage Services:

→ Availability, durability & data replication.

→ Availability — system uptime

↳ Based on SLA — how long system is operating.

↓
Service level agreement

→ durability — long term data protection.

↳ not being corrupted & lost in long run.

→ data replication : Replicate in all availability zones of a Region

S3 — 99.9999999% durability (11-9's)

99.99% availability (4-9's)

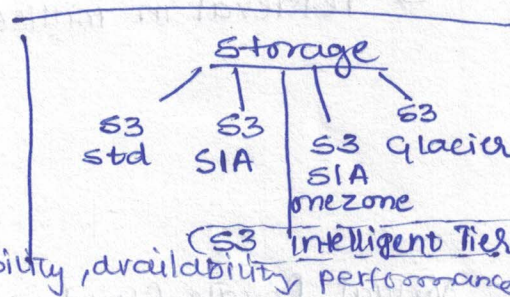
AKIS:

(3)

→ The objects are updated, updates are also replicated across all the AZs

Storage Classes:

→ Options to store: based on frequency



⇒ S3-Standard: frequently accessed, high durability, availability performance, low latency, high throughput
L 99.9999999% availability (5-9s), 99.999999999% durability (11-9s)
usecases: cloud, dynamic, mobile & etc., Big data

→ 0.000234/qb

⇒ S3-Standard-Infrequent Access: (multiple AZs) ↑ Availability Zones
L 99.9% (Avail), 99.99999999% (Durability)
3-9's, 9-9's
→ need to pay retrieval charge
→ low cost, high performance
→ used for long-term, backups, recovery (primary)
→ resilient against failure of AZs

⇒ S3-OneZone Infrequent Access: 80% cheaper than (S3-SIA)
L only in one AZ
→ lower cost,
→ for secondary backups

⇒ S3-Intelligent Tiering: Automatic classified into different storage classes.
L Optimized. (11-9 durability, 3-9 Availability)
→ monthly charge

⇒ S3-Glacier: - cheapest of all - long-term retention - deep Archives - cold storage
(11-9) durability (one or twice access / year)
→ for highly regulated.

Cheapest

⇒ 0.00004 \$ / GB

usecase: ?

→ used for long time archive such as patient data in healthcare systems.

S3 Glacier

Instant Retrieval:

→ retrieval in milliseconds

S3 Glacier flexible Retrieval

→ minutes to hours

• suited for backup & disaster recovery

Glacier Deep Archive

→ within 12 hours

→ cheapest

→ Att to magnetic tapes

↳ used for data archive large

Virtual Private Cloud :

|| VPC, internet gateway, subnet, Route table, Route 53, NACL

VPC: Virtual Network dedicated to our AWS account

VPC is isolated logical cloud/Network.

CIDR: Classless Inter-Domain Routing (10.0.0.0/16)

↳ IP addresses for each VPC.

VPC can span multiple AZs

Subnet & Gateways:

→ every subnet is given a CIDR

→ each subnets should be in one AZ only

→ subnets $\begin{cases} \text{Public} - \text{can be accessed through internet} \\ \text{private} \end{cases}$

↳ cannot be accessed Through internet

⇒ public subnets has internet gateways

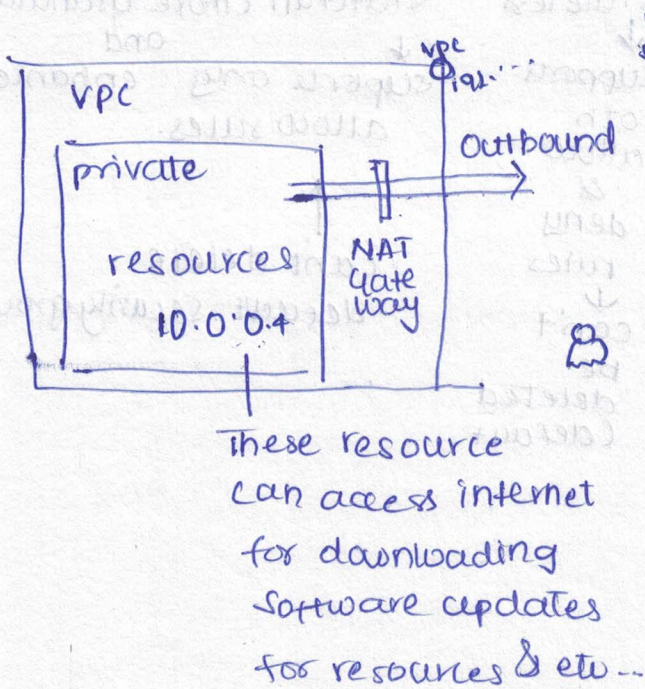
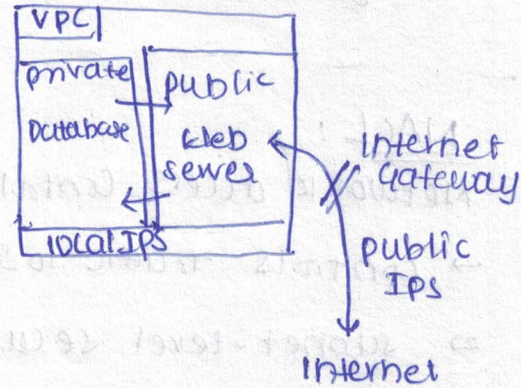
→ private subnets are internal processes, private subnets has route tables.

⇒ public subnets has to have both gateway - 0.0.0.0/0 and routers - local such as 10.0.0.0/16

⇒ private only has route table - 10.0.0.0/16.

→ To access the internet from the private subnet, we use NAT gateway (NAT - Network Address Translation). This gives access of internet to resource in private subnet possible, but their IP's are hidden, to make them secure.

- | public subnet | private Subnet |
|------------------------------------|-------------------------------------|
| → Contains web servers | → contains database servers |
| → can be accessed through internet | → cannot be |
| | → can only communicate indirectly - |



• NAT gateway doesn't allow inbound traffic by default. Hence the internet can't access private ressrcs, but private ressrc can access internet

• whereas Internet gateway, allows inbound access, and public subnet can be accessed using public IP address
 → IGW, does routing for the requests to the servers in public subnet.

Security Group & NACL

- both acts as firewalls
- control traffic flow
- Inbound ⇒ into network
- Outbound ⇒ out of network

NACL:

Network Access Control List

- controls traffic to & fro
- ⇒ subnet-level security
- each subnet should have NACL and only one NACL
- ⇒ As stateless, have a limited security

security group

- ⇒ AWS firewall solution
- filters incoming & outgoing traffic from EC2
- ⇒ instance level security
- ⇒ Independent rules for incoming & outgoing
- ⇒ can be applied to one or more EC2 instances.

NACL vs security group

Stateless

↓
support both allow & deny rules

↓
can't be deleted (default)

Stateful (more granular and enhanced)
↓
supports only allow rules.

↑
can't delete default security group.

Cloud Compute

- On demand computing service over internet - pay as you go
- Instance - virtual machines
 - Containers
 - serverless.

Virtual machine:

- VM are flexible & portable - Lowered downtime
- Cost saving
- Agility & Speed.
- Scalability.

EC2: Elastic compute cloud - scalable computing capacity.

→ flexible, IaaS, VMs

- Auto scaling available., supports Linux and also MacOS

Elements of EC2 [Instance - VM]

AMI - Amazon Machine Image - template containing OS & Softwares.

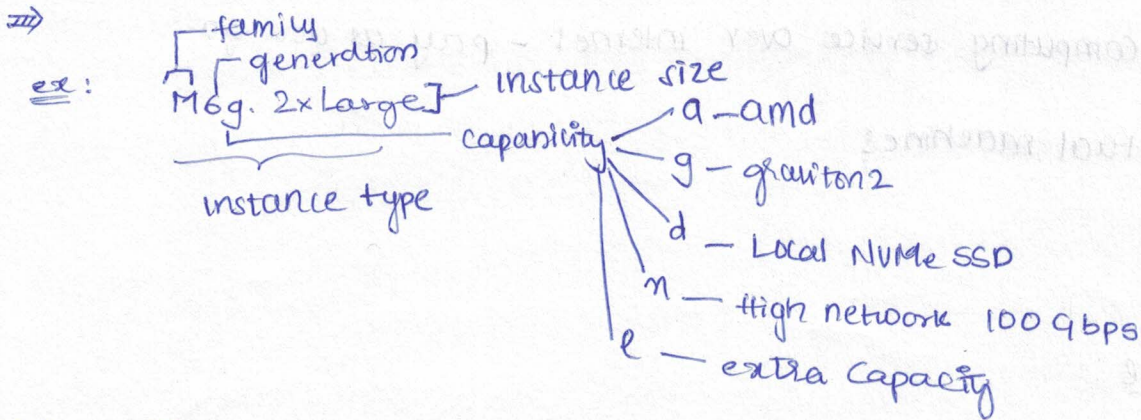
Key pair: (public - private keys)

Keeps
in instance

we download & connect to EC2

- security group - provides security for EC2 instance.
- Tags (key-value) Id's
- Cost allocation tags, to get segregation of costs in billing,

Naming Conventions



Instance Types :

EC2 - combine { CPU, Memory, Storage & Network Capacity }

EC2 instance grouped into families

- general-purpose —
 - compute-optimized — high performance CPU. { Batch process, ML, gaming, ad servers, ~~data~~ Scientific model }
 - memory-optimized — process large datasets — { opensrc database, in memory cache, realtime }
 - storage-optimized — { sequential read & write, low latency I/O reads }
 big data analytics
- used for databases and search engines, data analytics

Architecture

9

Load Balancers :

ELB - Elastic Load Balancers.

Auto scaling : → Elastic in nature

→ feature of cloud

→ automatically add & remove resources

→ Flexibility, Elasticity.

→ consistent performance @ low cost.

scaling in data centers traditionally

↳ manual & reactive

• Not Sustainable.

⇒ AWS Auto scaling : → monitors — Cloud watch (CPU, Network, memory)

↓
adjust Capacity.

↳ Threshold Based

(30% - 80%)

remove | add

→ Benefits : — scale up / down quickly

— Cost optimize

→ App performance maintained

⇒ AWS Auto scaling group : EC2

→ create a Auto group

→ Config — min
desired
max

↳ def : ~~group~~ logical collection of EC2 instances,
to scale dynamically

⇒ Also enables to have features like

• Health Check replacements

→ doesn't terminate
the min num of instance

• Scaling policy.

⇒ Desired Cap : → instances that are ideal, scaling starts by an event

$\text{min} \leq \text{Desired} \leq \text{max}$

⇒ max cap : not more than instances
created.

policies :

* Manual scaling

↳ manually change group
policies

* Dynamic Scaling

↳ response to changing demand

Dynamic scaling policies: Triggers by cloud watch.

- | | |
|-----------------------|---------------------------------------|
| simple / step scaling | Target tracking scaling |
| • based on metrics | • setting a target, tracking policies |

Schedule Scalling:

- manual.
- predictable load changes.
- known user patterns

* Predictive scaling - uses ML

- Advance forecasted load
- current and past patterns
- scales automatically

→ Beneficial for on / off traffic

##

Asg ↔ cloud watch

↙ scaling policy ↘

* Launch Configuration & Launch template:
(2014) ————— (recent)

→ define characteristics of EC2 instances.

→ config used by scaling groups to launch new instances.

⇒ AMI, instance type, security groups, roles, key value pairs, storage, user data

→ launch config → not editable.

⇒ Launch template: → extra features to launch config.

↳ multiple instance types

→ T2/T3 unlimited features

→ extra features

⇒ Recommended by AWS.

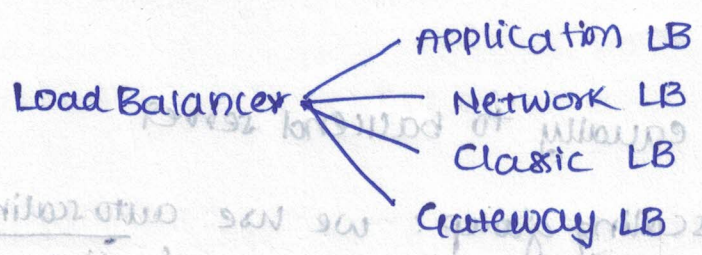
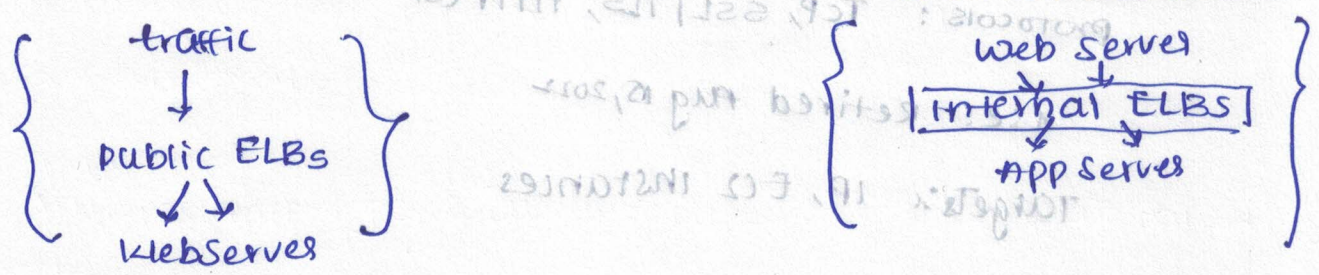
→ ASG uses template, whenever new instance has to be created.

Load Balancers :

- Network solⁿ, that routes server request to servers equally, to improve performances
- software solⁿ.
- scalability is also an issue for load balance, solⁿ → Elastic LB.

→ AWS ELB :

- distribute traffic b/w EC2 instances
- auto scaled.
- scaled along with increases w/ traffic increase
- Algo: Round Robin, least queue, etc...
- traffic's first point of contact ELB
- ELB checks EC2 instance health
- also used for internal routing too. } used in True decoupled architecture



LB :

Application LB : Layer 7

protocols : HTTP/HTTPS, gRPC

use : web apps, containers

Targets : IP, instances, Lambda.

Network LB : Layer 4

protocols : TCP, UDP, TLS

use : data flow, time sensitive, streaming apps

Targets : IP, instances, App LBs

Gateway LB : Layer 3 GW + Layer 4 LB

protocols : IP

use : 3rd party virtual apps

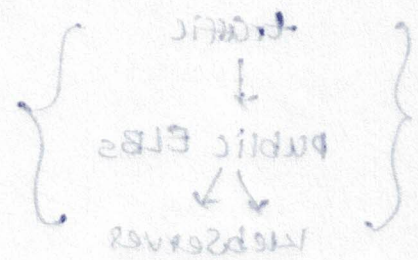
Targets : IP, instances

Classic LB : Layer 4 / Layer 7

protocols : TCP, SSL/TLS, HTTP(S)

use : Retired Aug 15, 2022

Targets : IP, EC2 instances



#Questions:

Why ELB : distribute traffic equally to backend server

To change instance type of Autoscaling group : we use auto scaling launch configuration

Benefit of ELB : maintain consistent performance

manual AS change : desired capacity to scale up

Services uses ELB : EC2 instances, etc..

Horizontal Scaling - doesn't change instance type but scales

AWS Database

data

- structured : stored in RDBS, has a data model
- semi-structured $\rightarrow$ JSON, XML (flexible structured data)
- unstructured : only has internal structure, don't have data model

Relational

- $\rightarrow$ Huge amount of structured data, has relationships b/w data
- $\rightarrow$ w/o redundancy, inconsistency
- $\rightarrow$ Manage, organised collection of data.
- $\rightarrow$ Data : structure $\rightarrow$ have a data model called Schema

AWS Sols : Amazon RDS

NoSQL : flexible, semi-structured

(non-relational)

- $\rightarrow$ documents, collections, databases, etc.
- $\rightarrow$ No schema is required
- $\rightarrow$ flexible number of attributes

* ElastiCache - used to easily deploy and run memcached & Redis protocol-compliant server
Amazon

On-premises VS cloud

↓
Networking
server
OS installation/patches
DB SW Install/patch
DB Backup
High Availability
scaling
App optimization

managed by customer
Network & server
given by AWS
Remaining managed by customer

managed by AWS
RDS
managed totally by AWS

Amazon RDS:

- Relational DB, managed on cloud
- Automated
- Fully managed & organised by AWS
- present in multi AZ
- maintenance window for patches
- support vertical & horizontal scaling
- * → cannot ssh into RDS instance

Vertical vs Horizontal

Vertical: more power
Horizontal: more machines & breaking the db and hosting on multiple

Features & Elements

- Availability & Durability w/ multi AZ redundancy (conn endpoint auto select)
- RDS instance types
- RDS Engine { PostgreSQL, MySQL, MariaDB, SQL Server, Amazon Aurora }
Compatible w/ PostgreSQL

→ RDS pricing: Instance Cost

on demand (1/hr) Reserved

Storage Cost

1GB/month

IO/month req

Amazon DynamoDB - NoSQL, scalable

↳ semi structure

↳ fully managed

→ encryption at rest/default

→ table can scale horizontally

- o tables - collections,
- o items,
- o attributes

- primary keys to ID collections & items
- secondary indexes - more query flexibility
- dynamoDB streams → capture modifications
- performance - throughput capacity
- performance meters - Console.
- on-demand backup capability / point in time recovery
- automatically delete expired items

use cases: mobile, web apps, game apps, edtech, IoT apps, etc...

* Amazon Aurora - MySQL, PostgreSQL compatible, open source cost effective

demo: RDS

- create subnet group first, add AZs to it
- keep private subnets primarily
- create either from backup from EC2 or new
- standard create / easy create (defaults) (preferable)
- ↓
- engine type: → versions too
- template: production / dev / free tier
 - creates standby
 - doesn't create standby
- Identifier - name of instance of RDS, not DB.
- credential settings.
- instance types: t3.micro
- storage type: gp2 (general purpose) / io1 (complex)
- AutoScale & max scale - 1000 GB (default)
- Availability (X) free tier

→ connectivity—VPC

→ don't allow public access

→ VPC—security group

⇒ DB authentication

- password
- pass + IAM
- pass + ~~kerberos~~ kerberos

→ DB name :

→ Enable automated backups → time window for backup.

→ cloudwatch : Normal vs Enhanced

from
hypervisor

from
Agent from OS level

→ logs → can send to cloud watch

→ delete protection.

→ create DB.

⇒ connection & EndPoint.

Quiz :

→ Responsible for patches for RDS ⇒ amazon product team

→ automanaged, NoSQL, AWS ⇒ DynamoDB

→ relational db proprietary to AWS, cloud optimized ⇒ Aurora

→ In-memory AWS DB, to reduce load off original DB (high perf, low lat) ⇒ Elastic cache

→ DB, SQL—Online Transaction Processing ⇒ RDS

→ NoSQL, no downtime, million/second, AWS ⇒ DynamoDB

→ fully managed, mongoDB ⇒ DynamoDB

→ RDS multi AZ ⇒ high Availability

RDS read replicas ⇒ Scalability.

⇒ Advanced

→ Storage Services

→ Elastic Block Storage [EBS]

→ persistent Block storage (volumes)

→ Hard drives for EC2 ~~etc~~

→ can keep file systems, OS, DB

→ Block storage - data into blocks/chunks

→ EBS is Network Attached Storage (NAS)

→ EBS can be connected / disconnected flexibly to multiple EC2

→ EBS IOPS

Features:

→ once EC2 to at a time (currently)

→ EBS restricted to one AZs

→ if want to move use snapshots

Types of EBS vols: 6 types

- General purpose SSD — gp2, gp3
 - provisioned IOPS SSD — io1, io2
 - Throughput optimized HDD (st1)
 - Cold HDD — sc1
- system boot volume supported.

→ Elastic File system: [EFS]

FS: Files named, stored, read managed.

CRUD of files managed by FS

organizer of storage

- cloud based, scalable FS
- free management, cost per GB, Network Attached Storage
- connect multiple EC2 inst at a time, across all AZs
ECS
EKS etc...
- EFS, compatible (w/ Linux)
- FSX for windows
- it's a managing service. — automatic service, no need to config
- EFS in one region, across all AZs

Route 53 :

DNS — domain Name system.

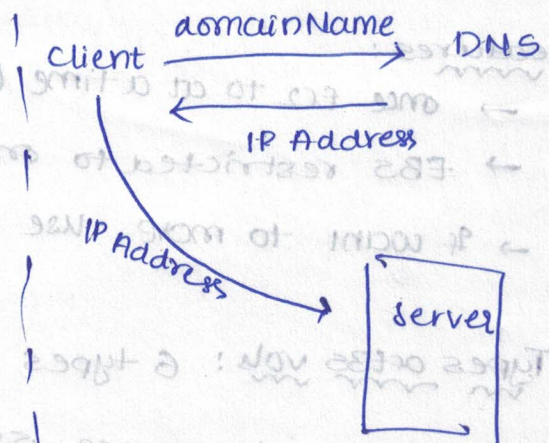
- Route 53 — 100% availability
- ↳ scalable DNS

- Domain Registrars — purchase DNS
- Transfer DNS

→ Hosted Zones

→ Health checks of resources

→ Traffic flow.



VPN & AWS Direct Connect

- on-premise + cloud combination.

Hence to connect, use ^{AWS} site-site VPN — Encrypted Conn.

config: VPN gateway, — at VPC
customer gateway — at client premise

- makes it private on public Internet

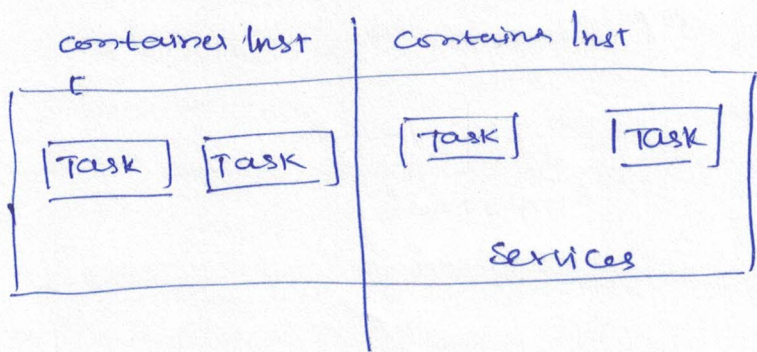
→ Direct Connect :

- High speed, low latency, physical connec, access public & private aws cloud from on-premise.
- virtual interface $\Rightarrow$ Direct Connect $\Rightarrow$ Router — (on-premise)
- Totally private

→ Elastic Container Service (ECS)

- Container — std unit of app code & dependencies (correct).
↳ docker platform.
- ECS — fully managed — container service orchestration of container
- Task Definition — json instruction about how to handle container
- Tasks —
- Task roles — permission given to container tasks

ECS Cluster



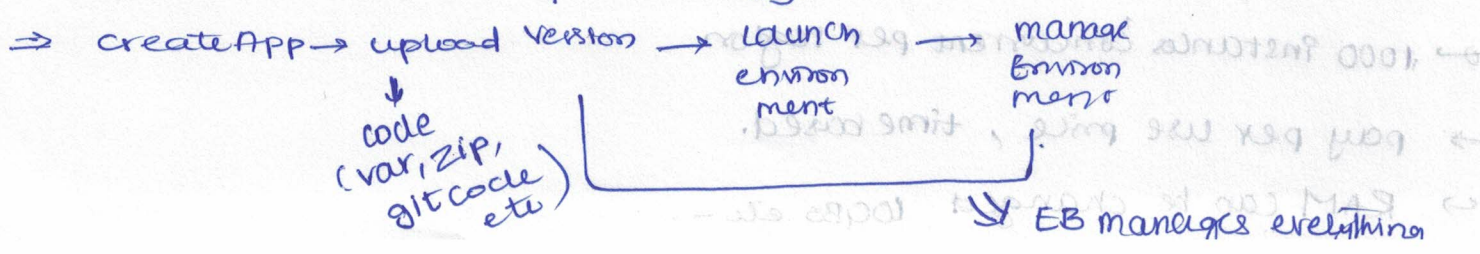
Launch types

- EC2 launch type : manual EC2 instance
- Fargate launch type : serverless

→ Elastic Beanstalk :

- deploy & scale web apps (easy-to-use)

EB



- Free service, but resources are billed, like database, S3, LB etc
- automatic deployment, dev only focus on code
- Support: : Java, .NET, PHP, Nodejs, Python, ruby, go, docker etc..
- Integrate VPC & launch resources
- also IAM
- also integrate CloudFront (uses S3 to store EB re)

Serverless

- No provisioned servers from customer
- no need to manage servers, provider manages.
- scales automatically.

Benefits:

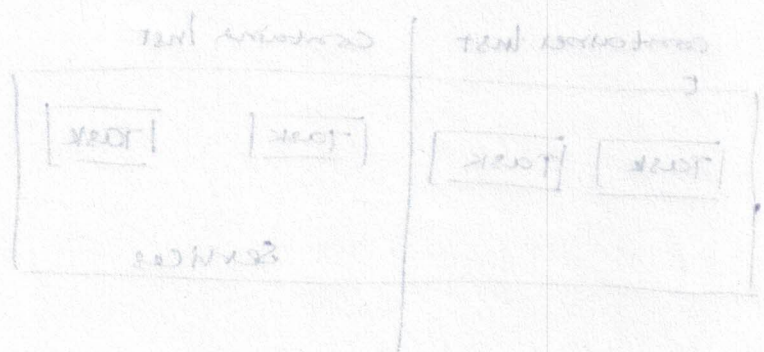
- no instance launch of resrcs
- automatic, high availability
- inexpensive
- scalable
- often uses container underneath

Lambda:

- serverless service
- function run on demand
- triggered by events.
- shorter execution
- max execution time of 15 mins (changes)

feature:

- automatic scaling, no policy
- 1000 instances concurrent per region
- pay per use price, time based.
- RAM can be changed 10GBs etc



→ usecases: upload process s3, API gateway, dynamoDB triggers etc.

→ SNS etc...

→ SQS queues, Cloudwatch events,

→ cloudwatch event triggers.

Amazon Elastic Cache :

→ cache - reuse frequency of data stores.

↳ reserved storage loc that holds temp data.

Elastic Cache → fully managed

↳ in-memory database

→ Improve performance of databases, application

→ doesn't require durability

→ Reduce database load.

→ low latency dB + non-durable)

Redshift :

→ data warehouse : large amount of data from various sources.

↳ Business data help org make better decision

→ Business Intelligence tools are helped by data warehouse.

Redshift — fast, fully managed, petabyte-scale data warehouse

10^{12} bytes

↳ simple & cost effective service for analysis of BI tools.

→ SQL based warehouse

→ uses EC2 instance, hence a need of launch config

→ pay as you go, incremental backups

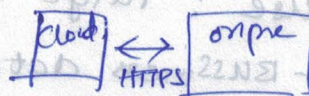
Amazon CloudFront

- A CDN: - distribution of static & dynamic web contents
- distance & latency $\propto$ $\frac{1}{\text{performance}}$
- cloudFront - caches on Edge location.
- ↳ if not in Edge loc, request original and caches
- useful for global access of websites

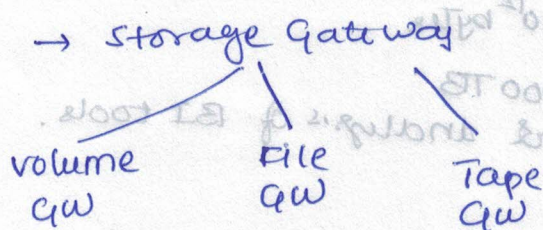
AKS snowball :

- migration service
- a petabyte scale data transport service
- transfer large amount to and out of cloud.
- snowball - a physical portable hard disk,
 - ↳ snowball ~~Edge~~ new version.
 - ↳ older
- Efficient
- computational capabilities too
- data processing can be done

AWS Storage Gateway :



- Hybrid option store
- integrate on premise datacentres to cloud, to make low latency



“we don't have to change alot to Int”

original code”

usecase: Backups to cloud

- Reduce local storage
- low latency access to data